

Operating Systems: Core Concepts

Chapter 1: Introduction to Operating Systems

An operating system (OS) is a system software that manages computer hardware, software resources, and provides common services for computer programs. It acts as an intermediary between users and the computer hardware. Major functions include process management, memory management, file system management, and device control.

The operating system provides a layer of abstraction that shields application programs from the complexity of the underlying hardware. Without an operating system, each application would need to implement its own hardware-level interactions, making software development considerably more difficult.

Types of Operating Systems

Batch Operating Systems process jobs without user interaction. Multiprogramming OS allows multiple programs to reside in memory simultaneously. Time-Sharing OS allocates CPU time to multiple users. Real-Time OS is used in systems where time constraints are critical, such as aircraft control or medical imaging devices. Distributed OS manages a group of distinct computers and presents them as a single coherent system.

Chapter 2: Process Management

A process is a program in execution. It includes the program code, current activity (program counter), a stack containing temporary data, and a data section containing global variables. The operating system manages processes by scheduling them on the CPU.

Process States

A process passes through several states during its lifetime. In the New state, the process is being created. In the Running state, instructions are being executed. In the Waiting state, the process is waiting for an event such as I/O completion. In the Ready state, the process is waiting to be assigned to a processor. In the Terminated state, the process has finished execution.

CPU Scheduling

CPU scheduling is the process of determining which process in the ready queue is allocated the CPU. Common scheduling algorithms include First-Come First-Served (FCFS), Shortest Job First (SJF), Round Robin (RR), and Priority Scheduling. Round Robin is widely used in time-sharing systems because it gives every process a fair share of CPU time.

Chapter 3: Memory Management

Memory management is one of the most critical responsibilities of an operating system. The OS must keep track of which parts of memory are in use and which are free. When a process requests memory, the OS must allocate it; when the process finishes, the OS must reclaim it.

Contiguous Memory Allocation

In contiguous memory allocation, each process is allocated a single contiguous section of memory. This can lead to fragmentation over time. External fragmentation occurs when there is enough total free memory but it is not contiguous. Internal fragmentation occurs when allocated memory is larger than what is needed.

Paging

Paging is a memory management scheme that eliminates the requirement for contiguous allocation of physical memory. Physical memory is divided into fixed-sized blocks called frames. Logical memory is divided into blocks of the same size called pages. When a process executes, its pages are loaded into available frames. The OS maintains a page table for each process that maps logical page numbers to physical frame numbers.

Chapter 4: Virtual Memory

Virtual memory is a memory management technique that provides an idealized abstraction of the storage resources actually available on a given machine. It creates the illusion for users of a very large primary memory. Virtual memory allows a computer to execute programs that are larger than the available physical RAM by using secondary storage as an extension of main memory.

The key benefit of virtual memory is that processes do not need to be entirely in physical memory to execute. Only the portions of a process that are currently being used need to reside in RAM. The remaining portions can be stored on disk and brought in only when needed.

Demand Paging

Demand paging is a technique used in virtual memory systems where pages are loaded into physical memory only when they are needed (demanded), rather than loading the entire program at once. This approach saves memory and time because unused pages are never loaded. When a page is needed but not in memory, a page fault occurs, triggering the OS to load that page from disk.

Page Faults

A page fault is a type of interrupt raised by hardware when a running program accesses a memory page that is mapped in the virtual address space but not loaded into physical memory. When a page fault occurs, the operating system must handle it by finding a free frame in physical memory, loading the required page from secondary storage, and then resuming the interrupted instruction.

Chapter 5: Page Replacement Algorithms

When a page fault occurs and all frames in physical memory are occupied, the OS must replace an existing page to make room for the new one. The algorithm used to decide which page to replace is called a page replacement algorithm. Choosing poorly can significantly degrade performance.

FIFO Page Replacement

First-In-First-Out (FIFO) is the simplest page replacement algorithm. In FIFO, the oldest page in memory is replaced first. While simple to implement, FIFO suffers from Belady's anomaly, where increasing the number of frames can paradoxically increase the page fault rate.

LRU Page Replacement

Least Recently Used (LRU) replacement replaces the page that has not been used for the longest period of time. LRU is based on the principle of temporal locality: if a page has not been used recently, it is unlikely to be used in the near future. LRU is more accurate than FIFO but is more complex to implement due to tracking usage history.

Thrashing

Thrashing is a condition where excessive paging activity degrades system performance significantly. When a process does not have enough frames to support its active working set, page faults occur at a very high rate. The OS spends more time handling page faults than executing actual process instructions. Thrashing can be detected by monitoring CPU utilization: if it drops sharply, thrashing is likely occurring.

Chapter 6: File Systems

A file system controls how data is stored and retrieved on storage devices. A file is a named collection of related data stored on secondary storage. File systems define the format and structure for storing, organizing, and accessing data on disk.

File Attributes

Files have attributes including name, type, location (pointer to file on device), size, protection (access rights), timestamps (creation, modification), and owner information. These attributes are maintained in the directory structure.

Directory Structures

Directories organize files within the file system. Single-level directories contain all files in a flat structure. Two-level directories have a separate directory for each user. Tree-structured directories allow arbitrary levels of subdirectories. Modern operating systems typically support tree-structured or acyclic graph directories.